

标签板子

状压 dp

子集枚举

```
1 for (int T = S; T; T = (T - 1) & S)
2 ...
```

容斥划分

按照二进制中 1 的个数决定正负。

SOS dp

就是超集和、子集和

已知集合的价值，计算所有超集的价值之和。

设 $dp[S][i]$ 表示集合 S 考虑前 i 位时，值为 0 的位可以自由变化后得到的价值和。

当第 i 位是 0 时：

$$dp[S][i] = dp[S][i-1] + dp[S|(1 \ll (i-1))][i-1]$$

比如 $dp[0010][3] = dp[0110][2] + dp[0010][2]$ 。

子集和同理，把 1 看作 0。

板子

```
1 for (int bit = 0; bit < m; bit++)
2   for (int s = 0; s < (1 << m); s++)
3     if (!(s & (1 << bit)))
4       dp[s] = dp[s] + dp[s | (1 << bit)];
```

这里原本 $dp[s]$ 是集合 s 的价值，通过维度空间压缩得到了超集和。

凸包

Andrew 凸包

来回扫两遍，左右转只需要保持一致即可。

板子

```
1 sort(points.begin(), points.end());
2
3 vector<Point> stk;
4
5 for (auto p : points)
6 {
```

```

7   while (stk.size() >= 2 &&
8       cross(stk.back() - stk[stk.size() - 2],
9       p - stk.back()) <= 0)
10      stk.pop_back();
11
12      stk.push_back(p);
13  }
14
15  int lower_size = stk.size();
16
17  for (int i = n - 2; i >= 0; i--)
18  {
19      auto p = points[i];
20
21      while (stk.size() > lower_size &&
22          cross(stk.back() - stk[stk.size() - 2],
23          p - stk.back()) <= 0)
24          stk.pop_back();
25
26      stk.push_back(p);
27  }
28
29  stk.pop_back();
30  // 注：这里最后得到的就是凸包顺序。如果 A、B、C 共线，
31  // 那么 B 会被排除，只得到极点；如果要保留 B，则需要额外处理 cross == 0。

```

回文字符串Manacher

作用

可以得到一个字符串每个位置的最大回文半径，复杂度为 $O(n)$ 。

大致思路

维护端点最靠右的回文串 center 和 right。当前处理 i，i 关于 center 的对称点 j 的回文半径是已知的，回文的信息也可以搬过来。

板子

```

1  vector<int> manacher(const string& s) {
2      string t;
3      t.reserve(2 * s.size() + 1);
4      for (char c : s) {
5          t.push_back('#');
6          t.push_back(c);
7      }
8      t.push_back('#');
9      int m = t.size();
10     vector<int> p(m);
11
12     int center = 0;
13     int right = 0; // 当前回文右端点的后一位
14
15     for (int i = 0; i < m; i++) {
16         if (i < right) {
17             int mirror = 2 * center - i;
18             p[i] = min(p[mirror], right - i);
19         } else {
20             p[i] = 1;
21         }
22
23         while (i - p[i] >= 0 &&
24             i + p[i] < m &&
25             t[i - p[i]] == t[i + p[i]]) {

```

```
26     p[i]++;
27     }
28
29     if (i + p[i] > right) {
30         center = i;
31         right = i + p[i];
32     }
33     }
34
35     return p;
36 }
```

位运算

对位运算的定义后面慢慢补充

关于牛客 8/7 场的 A 题，里面涉及一个贪心构造：给定一个 `ans`，里面为 1 的位是可以操作的；给定一个数组，问随意操作已知的位，能不能构造出单调不递减的数组？

核心就是 `getv(int last, int now)`：从 `ans` 的最高位开始遍历每一位 1，考虑什么时候这一位必须是 1。当后面的所有可操作位都是 1 还不够时，这一位一定是 1。

涉及到寻找最高位的 1：

```
1 63 - __builtin_clzll(x)
```

这里给出 `builtin` 的一系列用法：

```
1 __builtin_popcountll(x); // population count
2 __builtin_clzll(x);      // count leading zeros
3 __builtin_ctzll(x);      // count trailing zeros
4 __builtin_parityll(x);   // 判断 1 的个数的奇偶
```

做题记录

兜风

牛客 2026-09-01 点双连通分量 原题 20260901-01. cpp

题目大意

一次兜风就是从当前点出发，至少经过3个点，不经过重复点（除了当前点），最后回到当前点，问每个点最多能进行多少次兜风。

思路

有一个结论，如果当前点是 1，那么设它的邻居有 2, 3, 4, 5, 6。假设存在一个环经过 1, 2, 3，另外也有环经过 1, 3, 4 和 1, 4, 5，那么 2, 3, 4, 5 认为是一组可以两两搭配的集合，可以证明一定两两搭配。

可以看出，就是求点双。

代码

```
1  #include<bits/stdc++.h>
2  #define int long long
3  using namespace std;
4  const int N=2e5+7;
5  vector<pair<int,int> > g[N];
6  int dfn[N],low[N],f[N];
7  int tim;
8  int bianhao;
9  stack<int> stk;
10
11 void tarjan(int fa,int u)
12 {
13     low[u]=dfn[u]=++tim;
14     for(auto it:g[u])
15     {
16         int v=it.first;
17         if(v==fa)continue;
18
19
20         if(!dfn[v])
21         {
22             stk.push(it.second);
23             tarjan(u,v);
24             low[u]=min(low[u],low[v]);
25
26             if(low[v]>=dfn[u])
27             {
28                 while(1)
29                 {
30                     int i=stk.top();
31                     stk.pop();
32                     f[i]=bianhao;
33                     if(i==it.second)break;
34                 }
35                 bianhao++;
36             }
37         }
38         else if(dfn[v]<dfn[u])
39         {
40             stk.push(it.second);
41             low[u]=min(low[u],dfn[v]);
42         }
43     }
44 }
45
```

```

46 void solve()
47 {
48     int n,m;
49     cin>>n>>m;
50     tim=0;
51     bianhao=0;
52     while(!stk.empty())stk.pop();
53     for(int i=0;i<=n;i++)
54     {
55         g[i].clear();
56         dfn[i]=low[i]=f[i]=0;
57         bianhao=1;
58     }
59     for(int i=1;i<=m;i++)
60     {
61         int u,v;
62         cin>>u>>v;
63         g[u].push_back({v,i});
64         g[v].push_back({u,i});
65     }
66     for(int i=1;i<=n;i++)
67     {
68         if(dfn[i]==0)tarjan(0,i);
69     }
70     for(int i=1;i<=n;i++)
71     {
72         int ans=0;
73         map<int,int> mp;
74         for(auto it:g[i])
75         {
76             mp[f[it.second]]++;
77         }
78         for(auto it:mp)
79         {
80             ans+=it.second/2;
81         }
82         cout<<ans<<" \n"[i==n];
83     }
84 }
85
86 signed main()
87 {
88     ios::sync_with_stdio(false);
89     cin.tie(0);
90     int T;
91     cin>>T;
92     while(T--)
93     {
94         solve();
95     }
96 }
97 }

```

图穷匕首突起

码蹄集 2026-08-10 概率论 原题 20260810-01. cpp

题目大意

给定 n 根木棍的长度，小球每次落在某一根木棍的概率是 l_i/sum 。问，第一次连续三次落在同一根木棍的次数期望是多少？

思路

这是一个纯概率论问题。关键是状态要定义好+会写期望转移式。

定义期望： E_0 表示还没有开始， $E_{1,i}$ 表示最后连续落在第 i 根木棍一次， $E_{2,i}$ 表示两次。

期望转移式：

$$E_0 = 1 + \sum(i) p_i E_{1,i}$$

$$E_{1,i} = 1 + p_i E_{2,i} + \sum(j \neq i) p_j E_{1,j}$$

$$E_{2,i} = 1 + \sum(j \neq i) p_j E_{1,j}$$

关键就是前面有一个 1，然后按发生概率转移。

这里的 $\sum(j \neq i) p_j E_{1,j}$ 表示 $1 \leq j \leq n$ 且 $j \neq i$ 。

解方程，就是消元，然后同乘 p_i 累加 $E_{1,i}$ 。

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  #define double long double
5  const int mod=1e9+7;
6  const int N=5e5+7;
7  int l[N];
8
9  int qpow(int a,int p)
10 {
11     int ans=1;
12     while(p)
13     {
14         if(p&1)ans=ans*a%mod;
15         a=a*a%mod;
16         p>>=1;
17     }
18     return ans;
19 }
20
21 int inv(int a){
22     return qpow(a,mod-2);
23 }
24
25 signed main()
26 {
27     //cout<<log10(0ull-1ull);
28     int n;
29     cin>>n;
30     int sum=0;
31     for(int i=1;i<=n;i++)
32     {
33         cin>>l[i];
34         sum+=l[i];
35     }
36     int cnt=0;
37     for(int i=1;i<=n;i++)
38     {
39         int p=l[i]*inv(sum)%mod;
40         int tmpa=p*p%mod;
41         int tmpb=(1+p+tmpa)%mod;
42         tmpa=tmpa*p%mod;
43         cnt+=tmpa*inv(tmpb)%mod;
44         cnt%=mod;
45     }
46     cout<<inv(cnt);
47 }
```

天开武道会

牛客 2026-08-08 贪心 原题 20260808-02. cpp

题目大意

给定两个排列 ($n \leq 2e5$)。给定 x ，希望 x 能成为冠军。每次从这两个排列的左边选择第一个没有被淘汰的人，进行对战。如果选出同一个人，则发生死锁。如果一直没有死锁，最后剩下那个人就是冠军。如果 x 能成为冠军，问求淘汰顺序？

思路

这个题比较特别，它的贪心很有意思。

先求必要条件：

1. 对于 $1 \leq i \leq n$ ，如果两个排列前缀集合相同，那么一定会死锁。
2. 如果存在一个数，在两个排列中都出现在 x 后面，那么 x 一定不是冠军。

然后直接贪心构造：

设排列是 P, Q ，如果当前是 u 对战 v ，那么看 u 在 Q 、 v 在 P 里面的排名，选择排名靠前的淘汰。

比如当前队列是 $u, \dots, v$ 和 $v, \dots, u$ ，那么 v 的排名更靠前，淘汰 v 变成： $u, \dots$ 和 $\dots, u$ 。这个时候，因为我们继承了必要条件：没有相同的真前缀，继续检查发生变化的前缀。考虑第二个队列 $\dots, u$ ，如果没有到 u ，那一定不行；到了 u ，和原本就相差了 v ，也不会有相同的真前缀。

但是如果选择淘汰 u ：检查变化的区间， $\dots, v$ 和 $v, \dots$ 有可能发生相同前缀。

所以这个题就是：提出必要条件，贪心维护必要条件，最终变成充分条件。

但是这里看的是 u, v 的实时排名，显然不好维护。交了一发静态排名的，居然过了。

下面证明，如果有静态排名 $u < v$ ，则动态排名 $u \leq v$ ：

假设队列 1 是 $4, 5, 6, u, 1, 2, 3, v$ ，队列 2 是 $1, 2, 3, v, 4, 5, 6, u$ 。 $4, 5, 6$ 是队列 1 中被淘汰的， $1, 2, 3$ 同理。可以证明它们的位置肯定都在 u, v 中间（根据我们的贪心构造方法），所以不可能造成方向翻转。

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  #define double long double
5  const int N=2e5+7;
6  int a[N],b[N];
7  int isv[N];
8  vector<int> ans;
9  int mpa[N];
10 int mpb[N];
11 int n,x;
12
13 int check()
14 {
15     priority_queue<int,vector<int> > qa;
16     priority_queue<int,vector<int> > qb;
17     for(int i=1;i<n;i++)
18     {
19         qa.push(a[i]);
20         qb.push(b[i]);
21         while(!qa.empty()&&qa.top()==qb.top())
22         {
```

```

23     //cout<<qa.top()<<' '<<qb.top()<<endl;
24     qa.pop();
25     qb.pop();
26 }
27 if(qa.empty())return false;
28 }
29 //cout<<"??";
30 for(int i=1;i<=n;i++)
31 {
32     if(a[i]==x)
33     {
34         i++;
35         while(i<=n)isv[a[i++]]=1;
36     }
37 }
38 for(int i=1;i<=n;i++)
39 {
40     if(b[i]==x)
41     {
42         i++;
43         while(i<=n)
44         {
45             if(isv[b[i++]]=1)return false;
46         }
47     }
48 }
49 return true;
50 }
51
52 signed main()
53 {
54     cin>>n>>x;
55     for(int i=1;i<=n;i++)
56     {
57         cin>>a[i];
58         mpb[a[i]]=i;
59     }
60     for(int i=1;i<=n;i++)
61     {
62         cin>>b[i];
63         mpa[b[i]]=i;
64     }
65     if(!check())
66     {
67         cout<<"NO";
68         return 0;
69     }
70     for(int i=1;i<=n;i++)isv[i]=0;
71     int pa=1,pb=1;
72     while(a[pa]!=x||b[pb]!=x)
73     {
74         if(a[pa]==x)
75         {
76             ans.push_back(b[pb]);
77             isv[b[pb]]=1;
78         }
79         else if(b[pb]==x)
80         {
81             ans.push_back(a[pa]);
82             isv[a[pa]]=1;
83         }
84         else
85         {
86             if(mpa[a[pa]]<mpb[b[pb]])
87             {
88                 ans.push_back(a[pa]);
89                 isv[a[pa]]=1;
90             }
91             else
92             {
93                 ans.push_back(b[pb]);
94                 isv[b[pb]]=1;

```



```

95     }
96     }
97     while(pa<=n&&isv[a[pa]])pa++;
98     while(pb<=n&&isv[b[pb]])pb++;
99     }
100    int sz=ans.size();
101    cout<<"YES\n";
102    for(int i=0;i<sz;i++)
103    {
104        if(i)cout<<' ';
105        cout<<ans[i];
106    }
107 }

```

天使领域破解

牛客 2026-08-08 位运算 原题 20260808-01.cpp

题目大意

给定 n 个数 ($2e5$)，可以进行若干次操作，每次可以选择一个 k ，选择任意个元素和 k 做异或，使得整个数组单调不下降，求最终 k 的总和最小值。

思路

显然很容易可以发现，必须要按位考虑，而且 k 每次最好只有一位是 1。

然后按最高位考虑，希望迫不得已才选择操作这一位。

什么时候是必须操作的？一开始认为这是一个神秘的区间分割、贪心分配，比如第 4 位是 0, 0, 1, 1, 1，那么只需要考虑两个子区间；如果某一位选择操作，那么每个子区间可以再任意分割成子区间。这显然很复杂，很多时候位运算不会涉及到这种。

其实目前的问题是：“如何判断当前位一定要操作？”其实可以贪心构造，每个位置选择恰好大于上一个数的最小值。

代码

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  #define double long double
5  const int N=2e5+7;
6  int a[N];
7  int ans;
8
9  inline int get(int last,int now)
10 {
11     //cout<<last<<' '<<now<<' ';
12     int k=ans;
13     while(k!=0)
14     {
15         int mask=63 - __builtin_clzll(k);
16         mask=1ll<<mask;
17         k=k^mask;
18         if((now|k)<last)now|=mask;
19     }
20     if(now<last)now+=1;
21     // cout<<now<<endl;
22     return now;
23 }
24
25 void solve()
26 {

```

```

27     int n;
28     cin>>n;
29     ans=0;
30     for(int i=1;i<=n;i++)cin>>a[i];
31     for(int i=30;i>=0;i--)
32     {
33         int last=0;
34         int mask=(1<<i)-1;
35         mask|=ans;
36         mask=~mask;
37         //cout<<"i"<<' '<<bitset<10>(mask)<<endl;
38         for(int j=1;j<=n;j++)
39         {
40             int now=a[j]&mask;
41             now=get(last,now);
42             // cout<<i<<' '<<now<<endl;
43             if(now==1)
44             {
45                 ans=ans|(1<<i);
46                 break;
47             }
48             last=now;
49         }
50     }
51     cout<<ans<<'\n';
52 }
53
54 signed main()
55 {
56     int t;
57     cin>>t;
58     while(t--)
59     {
60         solve();
61     }
62 }

```

悬赏千金抓人

码蹄集 2026-08-03 单调栈 原题 20260803-02.cpp

题目大意

给定 $n*m$ 矩阵，每个位置给定价值。给定 k ，求总和 $\geq k$ 的矩形最小面积（一个位置面积是1）。

思路

显然直接暴力是 $O(n^4)$ ，会爆，而这个题给的是 $O(n^3)$ 不会爆。希望枚举上下行，然后子问题转化为求和满足 $\geq k$ 的最短子数组。

对于这个子问题，观察前缀和 pre：如果是 3, 1, 6，那么我们不用考虑 3。假设 3 和后面的 6 构成 $6-3 \geq k$ ，但是 3 前面有更小的 1，则 $6-1 \geq k$ 一定成立，且长度更短。于是可以维护单调的左端点。

但是枚举完左端点，直接继续枚举右端点， $O(n^2)$ 还是炸。于是继续观察，发现：如果当前队列是 1, 2, 3，当前点是 4，假如 1, 4 满足了，那么后面的所有右端点都不需要考虑，因为不可能有更优的答案。

总的来说就是两个小思维点。

代码

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     #define int long long
4     #define double long double
5     const int N=307;

```

```

6  int a[N][N];
7  int pr[N][N];
8  int pre[N];
9  int n,m,k;
10
11 int solve(int len)
12 {
13
14     int ans=1e18;
15     deque<int> dq;
16     dq.push_back(0);
17     for(int i=1;i<=m;i++)
18     {
19         while(!dq.empty()&&pre[i]<=pre[dq.back()])dq.pop_back();
20         while(!dq.empty()&&pre[i]-pre[dq.front()]>=k)
21         {
22             ans=min(ans,(i-dq.front())*len);
23             dq.pop_front();
24         }
25         dq.push_back(i);
26     }
27     return ans;
28 }
29
30 void work()
31 {
32     cin>>n>>m>>k;
33     for(int i=1;i<=n;i++)
34         for(int j=1;j<=m;j++)
35             cin>>a[i][j];
36     for(int i=1;i<=n;i++)
37     {
38         for(int j=1;j<=m;j++)
39         {
40             pr[i][j]=pr[i-1][j]+pr[i][j-1]-pr[i-1][j-1]+a[i][j];
41         }
42     }
43     int ans=1e18;
44     for(int i=1;i<=n;i++)
45     {
46         for(int j=i;j<=n;j++)
47         {
48             for(int k=1;k<=m;k++)
49             {
50                 pre[k]=pr[j][k]-pr[i-1][k];
51             }
52             // if(j==i+1)
53             // {
54             //     cout<<"i:"<<i<<endl;
55             //     for(int k=1;k<=m;k++)
56             //     {
57             //         cout<<pre[k]<<" \n"[k==m];
58             //     }
59             // }
60             ans=min(solve(j-i+1),ans);
61         }
62     }
63     if(ans==1e18)cout<<1<<"\n";
64     else cout<<ans<<"\n";
65 }
66
67 signed main()
68 {
69     int t;
70     cin>>t;
71     while(t-->0)work();
72 }

```

聂菽认尸殉弟

码蹄集 2026-08-03 并查集 / 离线查询 原题 20260803-01.cpp

题目大意

给定 n 个点， m 条边，每个点有访问阈值 a_i 。有 q 次查询，每次给出 $u\ x$ ，表示从 u 出发，能力为 x 能访问多少个点？（即使 $a_u > x$ 也能访问初始点 u ）

思路

显然普通的 BFS、DFS 不能处理。很多图论的题，并查集还是很管用的。需要配合离线查询，按照访问能力 x 排序，这样能访问的点是只增不减的，于是可以维护并查集，表示当前访问能力能处理什么样的集合。

代码里面，需要注意的是按照访问阈值排序点的时候，分清 u 和 $w[u]$ ；排序查询的时候，是 $\text{sort}(qr + 1, qr + 1 + q, \dots)$ ，而不是 $qr + 1 + n$ 。

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  const int N=5e5+7;
5  vector<int> g[N];
6  int w[N];
7  int a[N];
8  int fa[N];
9  int sz[N];
10 int isv[N];
11
12 int find(int x)
13 {
14     if(fa[x]==x)return x;
15     fa[x]=find(fa[x]);
16     return fa[x];
17 }
18
19 void merge(int i,int j)
20 {
21     int ii=find(i);
22     int jj=find(j);
23     if(ii==jj)return;
24     if(sz[ii]>sz[jj])
25     {
26         fa[jj]=ii;
27         sz[ii]+=sz[jj];
28     }
29     else
30     {
31         fa[ii]=jj;
32         sz[jj]+=sz[ii];
33     }
34 }
35
36 struct query
37 {
38     int u,x;
39     int i;
40     int ans;
41 };
42 query qr[N];
43
44 signed main()
45 {
46     ios::sync_with_stdio(false);
47     cin.tie(0);
```

```

48     cout.tie(0);
49
50     int n,m;
51     cin>>n>>m;
52     for(int i=1;i<=n;i++)cin>>w[i];
53     for(int i=1;i<=n;i++)
54     {
55         a[i]=i;
56         fa[i]=i;
57         sz[i]=1;
58     }
59     sort(a+1,a+1+n,[](const int&l,const int &r)
60     {
61         return w[l]<w[r];
62     });
63     //for(int i=1;i<=n;i++)cout<<a[i]<<endl;
64
65     while(m--)
66     {
67         int u,v;
68         cin>>u>>v;
69         g[u].push_back(v);
70         g[v].push_back(u);
71     }
72     int q;
73     cin>>q;
74     for(int i=1;i<=q;i++)
75     {
76         qr[i].i=i;
77         cin>>qr[i].u>>qr[i].x;
78     }
79     sort(qr+1,qr+1+q,[](const query&l,const query&r)
80     {
81         return l.x<r.x;
82     });
83
84     //for(int i=1;i<=q;i++)cout<<qr[i].u<<' '<<qr[i].x<<endl;
85
86     int p=1;
87     for(int i=1;i<=q;i++)
88     {
89         while(p<=n&&w[a[p]]<=qr[i].x)
90         {
91             int u=a[p];
92             for(int v:g[u])
93             {
94                 if(!isv[v])
95                 {
96                     merge(v,u);
97                 }
98             }
99             isv[u]=1;
100             //cout<<"p:"<<p<<' '<<u<<endl;
101             p++;
102         }
103         //cout<<"find"<<find(qr[i].u)<<endl;
104         //for(int i=1;i<=n;i++)cout<<fa[i]<<" \n"[i==n];
105         //cout<<"qr"<<qr[i].x<<endl;
106         qr[i].ans=sz[find(qr[i].u)];
107     }
108     sort(qr+1,qr+1+q,[](const query&l,const query&r)
109     {
110         return l.i<r.i;
111     });
112     for(int i=1;i<=q;i++)cout<<qr[i].ans<<' \n';
113 }

```

评测排序

牛客 2026-07-29 状压dp / SOS dp 原题 20260729-02.cpp

题目大意

给定 n 个程序，有 m 个评测点，每个点每个程序给定是否通过、评测耗时。拒绝之后的评测都不需要跑，问如何安排评测点顺序使得总时间最小。 $m \leq 20$, $n \leq 3e5$

思路

难点：

1. 发现 $m=20$ ，往状压 DP 想。
2. 会算 $cost[S][i]$ ，也就是 SOS DP。已知集合的价值，求每个集合对应的所有超集的价值之和。

SOS DP，超集和、子集和。

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  #define double long double
5  const int N=2e4+7;
6  const int M=1<<20;
7  int a[N][22];
8  int f[M][22];
9  int mk[N];
10 int dp[M];
11
12 signed main()
13 {
14     ios::sync_with_stdio(false);
15     cin.tie(0);
16     // cout<<pow(2,20);
17     int n,m;
18     cin>>n>>m;
19     for(int i=1;i<=n;i++)
20     {
21         for(int j=1;j<=m;j++)
22         {
23             cin>>a[i][m-j+1];
24         }
25         for(int j=1;j<=m;j++)
26         {
27             char c;
28             cin>>c;
29             mk[i]<=1;
30             if(c=='A')
31             {
32                 mk[i]=1;
33             }
34         }
35         //cout<<mk[i]<<endl;
36         for(int j=1;j<=m;j++)
37         {
38             f[mk[i]][j]+=a[i][j];
39         }
40     }
41
42     int mm=1<<m;
43     for(int bit=0;bit<m;bit++)
44     {
45         for(int mask=0;mask<mm;mask++)
46     {
```

```

47         if(!(mask&(1<<bit)))
48         {
49             for(int i=1;i<=m;i++)
50             {
51                 f[mask][i]+=f[mask|(1<<bit)][i];
52             }
53         }
54     }
55 }
56 // for(int i=0;i<mm;i++)
57 // {
58 //     for(int j=1;j<=m;j++)cout<<f[i][j]<<" \n"[j==m];
59 // }
60
61 for(int i=1;i<M;i++)dp[i]=1e18;
62 for(int i=0;i<mm;i++)
63 {
64     for(int j=0;j<m;j++)
65     {
66         int mask=~(1<<j);
67         if(i&(1<<j))
68         {
69             dp[i]=min(dp[i],dp[i&mask]+f[i&mask][j+1]);
70             //cout<<i<<" "<<(i&mask)<<" "<<j<<endl;
71         }
72     }
73 }
74 cout<<dp[mm-1]<<endl;
75 }

```

字典最小博弈

牛客 2026-07-29 博弈 原题 20260729-01.cpp

题目大意

爱丽丝和鲍勃在玩一个游戏。

最初，有一个空序列。给他们一个整数 n ，他们将构造一个长度为 n 的排列 p 。

爱丽丝先下。每次移动时，当前棋手都会从 1 到 n 之间选择一个之前没有选择过的整数，并将其附加到序列的末尾。经过恰好 n 步之后，序列就变成了 $1, 2, \dots, n$ 的排列组合 p 。

排列组合 $p=(p_1, p_2, \dots, p_n)$ 的循环移动是选择索引 i 而得到的序列。 ($1 \leq i \leq n$) 并写入 $(p_i, p_{i+1}, \dots, p_n, p_1, p_2, \dots, p_{i-1})$ 。例如， $(2, 3, 1)$ 的循环移位是 $(2, 3, 1)$ 、 $(3, 1, 2)$ 和 $(1, 2, 3)$ 。

对于一个排列组合 p ，定义 $f(p)$ 为 p 的词典最小循环移位。

爱丽丝希望在词典上最小化 $f(p)$ ，而鲍勃希望在词典上最大化 $f(p)$ 。

假设双方都以最优方式下棋，请确定所得到的排列组合 $f(p)$ 。

思路

显然这个题很有打表的套路。它很可能是个连蒙带猜的题。

于是直接用 $\min(\max(\dots))$ 打表，基本没有太多的技巧。

难道博弈题就是这种神秘的规律？

代码

```

1 #include <bits/stdc++.h>
2 using namespace std;

```

```

3  #define int long long
4  const int N=5e5+7;
5  int a[N];
6
7
8  //string gets(vector<int>&v)
9  //{
10 // string ans;
11 // int flag=0;
12 // for(int i:v)
13 // {
14 //     if(i==1)flag=1;
15 //     if(flag)ans=ans+char(i+'0');
16 // }
17 // for(int i:v)
18 // {
19 //     if(i==1)break;
20 //     ans=ans+char(i+'0');
21 // }
22 // return ans;
23 //}
24 //
25 //bool isv(vector<int>&v,int i)
26 //{
27 // for(int j:v)if(i==j)return true;
28 // return false;
29 //}
30 //
31 //string solve(int k,vector<int>&v,int n)
32 //{
33 // if(k>n)return gets(v);
34 // string ans;
35 // int fir=1;
36 // if(k%2)
37 // {
38 //     for(int i=1;i<=n;i++)
39 //     {
40 //         if(isv(v,i))continue;
41 //         v.push_back(i);
42 //         if(fir)
43 //         {
44 //             fir=0;
45 //             ans=solve(k+1,v,n);
46 //         }
47 //         else
48 //         {
49 //             string tmp=solve(k+1,v,n);
50 //             if(ans>tmp)ans=tmp;
51 //         }
52 //         v.pop_back();
53 //     }
54 // }
55 // else
56 // {
57 //     for(int i=1;i<=n;i++)
58 //     {
59 //         if(isv(v,i))continue;
60 //         v.push_back(i);
61 //         if(fir)
62 //         {
63 //             fir=0;
64 //             ans=solve(k+1,v,n);
65 //         }
66 //         else
67 //         {
68 //             string tmp=solve(k+1,v,n);
69 //             if(ans<tmp)ans=tmp;
70 //         }
71 //         v.pop_back();
72 //     }
73 // }
74 // return ans;

```



```

75 //}
76 //
77 //
78 //void out(string s)
79 //{
80 // for(char c:s)cout<<c-'0'<<' ';
81 // cout<<endl;
82 //}
83
84 signed main()
85 {
86 // for(int i=2;i<=11;i+=2)
87 // {
88 //     vector<int> v;
89 //     out(solve(1,v,i));
90 // }
91     int t;
92     cin>>t;
93     while(t--)
94     {
95         int n;
96         cin>>n;
97         if(n%2)
98         {
99             a[1]=1;
100             a[3]=2;
101             int st1=n,st2=3;
102             for(int i=n;i>=2;i--)
103             {
104                 if(i==3)continue;
105                 if(i%2)a[i]=st1--;
106                 else a[i]=st2++;
107             }
108             for(int i=1;i<=n;i++)cout<<a[i]<<" \n"[i==n];
109         }
110         else
111         {
112             a[1]=1;
113             int st1=n,st2=2;
114             for(int i=n;i>=2;i--)
115             {
116                 if(i%2==0)a[i]=st1--;
117                 else a[i]=st2++;
118             }
119             for(int i=1;i<=n;i++)cout<<a[i]<<" \n"[i==n];
120         }
121     }
122 }

```